



# CSS3: Layout, Box Model & Responsive Design

Web Design and Programming

**M.Sc. Le Duy Khanh**

Faculty of Data Science and Artificial Intelligence



# Session Outline

- 1. CSS Foundations & Selectors**
- 2. The Box Model**
- 3. Flexbox & Grid**
- 4. Responsive Web Design**
- 5. Review & Summary**



# Learning Objectives

By the end of this session, you will be able to:

1. Explain CSS selectors, cascade, inheritance, and specificity.
2. Apply the Box Model to calculate element spacing and total size.
3. Build one-dimensional layouts with Flexbox and understand when Grid is better.
4. Use media queries to create responsive views for desktop and mobile.
5. Build a responsive KPI dashboard card layout for DS and AI reporting.



# How to Approach This Session

- CSS is the **presentation layer** – it styles the HTML skeleton you built in Week 2.
- Focus on the **Box Model and Flexbox**: these cover 80% of practical use cases.
- Think like a programmer, not a graphic designer: CSS is a **logic system** with rules, priorities, and cascades.
- For DS/AI students, the most relevant skill is **dashboard layout** – KPI cards, charts side by side, responsive tables.



# Environment Check

- Open **VS Code** with the **Live Server** extension running.
- Create `style.css` and link it to your `index.html` from Week 2: `<link rel="stylesheet" href="style.css">`
- Confirm that changing a color in `style.css` reflects immediately in the browser.



# CSS Foundations & Selectors

---

# What Is CSS?

- CSS = **Cascading Style Sheets** – describes how HTML elements are displayed.
- A CSS rule consists of a **selector** and a **declaration block**.
- Declarations follow the pattern: property: value;
- CSS can be: **inline**, in a `<style>` tag, or in an **external** .css file (preferred).



# CSS Rule Structure

```
1 /* Element selector */
2 p { color: #333; }
3
4 /* Class selector */
5 .card { background-color: #f5f5f5; }
6
7 /* ID selector */
8 #header { color: #cc3300; }
```

- Element: targets all matching tags.
- Class (.name): targets multiple elements sharing a style.
- ID (#name): targets a **single unique** element.





# Cascade, Specificity & Inheritance

- **Cascade:** when rules conflict, the browser picks the winner by **priority then source order**.
- **Specificity order:** inline styles > #id > .class > element.
- **Inheritance:** some properties (font-family, color) flow from parent to child automatically.

**Exam cue:** if a `<h1>` matches `h1`, `.title`, and `#header` simultaneously, the `#header` rule wins every time.

# Specificity in Practice

```
1 h1      { color: #333;    } /* specificity: 0,0,1 */
2 .title  { color: #0055aa; } /* specificity: 0,1,0 */
3 #header { color: #cc3300; } /* specificity: 1,0,0 */
```

- The three rules compete for the same element.
- #header wins – highest specificity (1,0,0).
- If two rules have equal specificity, the **later one wins** (source order).

# Combining Selectors

```
1 /* Descendant: .card inside .dashboard */
2 .dashboard .card { border: 1px solid #ddd; }
3
4 /* Pseudo-class: hover state */
5 .card:hover { background-color: #e8f0fe; }
6
7 /* Multiple targets */
8 h1, h2, h3 { font-weight: bold; }
```

- Descendant selectors let you style elements **by context**.
- Pseudo-classes like :hover, :focus add interactive styling.



## Further Reading (Part 1)

### Optional – self-study handout

- CSS attribute selectors: `[type="email"]`.
- Pseudo-elements: `::before`, `::after`.
- CSS specificity calculator: [specificity.keegan.st](http://specificity.keegan.st).
- Source: MDN – *CSS Selectors, Cascade and Inheritance*.



# Lab 1: Selectors and Styling

## Hands-on practice

Add a `style.css` file and link it to your Week 2 page. Then:

- Style the `<h1>` element with a font color and size.
- Create a `.highlight` class and apply it to one paragraph.
- Create an `#main-header` ID rule and observe it override the element rule.
- *Finished early?* Add a `:hover` color change to any link.



# The Box Model

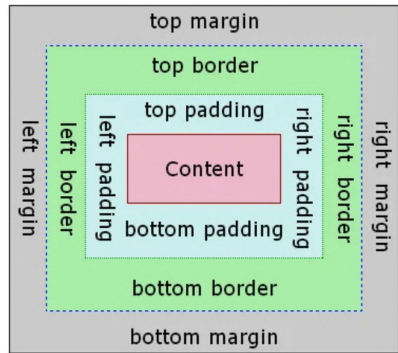
---

# Every Element Is a Box (Critical Topic)

Every HTML element occupies a rectangular box made of four layers:

1. **Content** – the actual text or image area.
2. **Padding** – transparent space *inside* the border.
3. **Border** – the visible or invisible edge line.
4. **Margin** – transparent space *outside* the border.

Understanding this is essential for **any layout work** and is a high-frequency exam topic.





# Box Model Width Calculation

Default model (box-sizing: content-box):

$$\text{Total width} = \text{content} + 2 \times \text{padding} + 2 \times \text{border} + 2 \times \text{margin}$$

## MCQ-style example:

- width: 200px, padding: 10px, border: 2px, margin: 5px
- Total =  $200 + 20 + 4 + 10 = \mathbf{234\text{px}}$

**Tip:** memorize the formula – it appears frequently in exams.



# Padding vs. Margin

```
1 .card {  
2   padding: 16px;           /* space inside , between text and border */  
3   border: 2px solid #ccc;  
4   margin: 12px;           /* space outside , between cards */  
5 }
```

|            | Padding             | Margin                     |
|------------|---------------------|----------------------------|
| Position   | inside the border   | outside the border         |
| Background | inherits element bg | always transparent         |
| Exam cue   | inner spacing       | outer spacing / separation |

## box-sizing: border-box

```
1 /* Apply globally --- recommended in every project */
2 *,
3 *::before ,
4 *::after {
5     box-sizing: border-box;
6 }
```

- With border-box: padding and border are **included in** the declared width.
- A width: 200px element with padding: 10px still occupies **200px**, not 220px.
- Greatly reduces layout surprises – use it on every project.



## Further Reading (Part 2)

### Optional – self-study handout

- Margin collapsing: adjacent vertical margins collapse to the larger one.
- Outline vs. border (outline does not affect layout).
- Source: MDN – *The CSS Box Model*; w3schools – *CSS Box Model*.



## Lab 2: Box Model Exploration

### Hands-on practice

Create three `<div>` elements styled as data cards:

- Add padding, border, and margin individually.
- Use browser DevTools to inspect the computed box sizes.
- Toggle box-sizing: border-box and observe the width change.
- *Finished early?* Calculate the total width manually and verify in DevTools.



# Flexbox & Grid

---

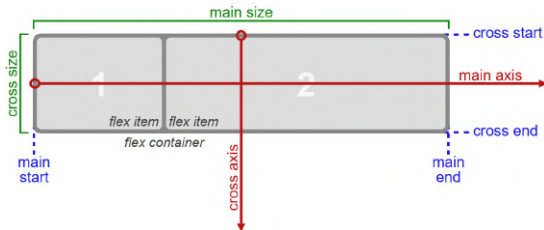


# Modern CSS Layout Tools

- Before Flexbox/Grid, layouts relied on float hacks – error-prone and hard to maintain.
- **Flexbox:** one-dimensional alignment – ideal for rows of cards, navigation bars, form rows.
- **CSS Grid:** two-dimensional layout – ideal for full-page regions, dashboards with fixed rows and columns.
- **Decision rule:** Flexbox for linear flow; Grid for structured 2D grids.

# Flexbox Mental Model

- Enable with display: flex on the **container**.
- The container controls how children (flex items) are positioned.
- **Main axis** (default: horizontal): controlled by justify-content.
- **Cross axis** (default: vertical): controlled by align-items.
- Items can wrap to the next line with flex-wrap: wrap.



# Flexbox Container Properties

```
1 .dashboard {  
2   display: flex;  
3   flex-direction: row;           /* row | column */  
4   flex-wrap: wrap;              /* wrap onto next line if needed */  
5   justify-content: flex-start; /* main-axis alignment */  
6   align-items: stretch;        /* cross-axis alignment */  
7   gap: 16px;                   /* space between items */  
8 }
```

- justify-content: flex-start, center, flex-end, space-between, space-around.
- align-items: stretch, center, flex-start, flex-end.



# Flexbox Item Properties

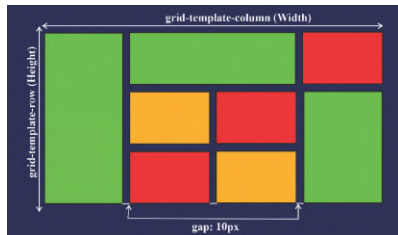
```
1 .card {  
2   flex: 1 1 220px;  
3   /* shorthand: flex-grow flex-shrink flex-basis */  
4   padding: 14px;  
5   border: 1px solid #d4d4d4;  
6   border-radius: 10px;  
7 }
```

- flex-grow: how much extra space the item claims.
- flex-shrink: how much the item shrinks when space is tight.
- flex-basis: the starting size before growing/shrinking.
- flex: 1 1 220px means: *start at 220px, grow and shrink as needed.*

# CSS Grid Overview

```
1  .layout {  
2      display: grid;  
3      grid-template-columns: 1fr 1fr 1fr; /* 3 equal columns */  
4      gap: 16px;  
5  }
```

- 1fr = one fraction of available space.
- Grid places items into explicit rows and columns.
- Ideal for: full-page layouts, image galleries, admin panels.
- Both Grid and Flexbox can be used **together** on the same page.





# Flexbox vs. Grid Quick Reference

|                 | Flexbox                  | Grid                    |
|-----------------|--------------------------|-------------------------|
| Dimensions      | 1D (row or column)       | 2D (rows and columns)   |
| Best for        | cards, nav, form rows    | page regions, galleries |
| Item sizing     | flexible, content-driven | precise, grid-aligned   |
| Browser support | excellent                | excellent               |



## Further Reading (Part 3)

### Optional – self-study handout

- Flexbox: order, align-self, flex-flow shorthand.
- Grid: grid-area, auto-fill, minmax().
- Interactive tools: [flexboxfroggy.com](https://flexboxfroggy.com), [cssgridgarden.com](https://cssgridgarden.com).
- Source: MDN – *CSS Flexible Box Layout*; *CSS Grid Layout*.



## Lab 3: Flexbox Card Row

### Hands-on practice

Build a `.dashboard` container with four `.card` children:

- Apply `display: flex` and `gap` on the container.
- Set `flex: 1 1 200px` on each card so they share space equally.
- Add padding, border-radius, and a background color.
- *Finished early?* Try changing `justify-content` to `space-between` and observe the difference.



# Responsive Web Design

---

# What Is Responsive Web Design (RWD)?

- One HTML document, one CSS file – **automatically adapts** to any screen size.
- Key principle: use relative units and conditional CSS rules, not fixed pixel widths.
- Why it matters for DS/AI: dashboards must work on **desktop monitors, tablets, and phones**.





# Viewport Meta Tag – Required

1 `<meta name="viewport" content="width=device-width, initial-scale=1">`

- Without this, mobile browsers **pretend to be desktop** and shrink everything.
- This tag was already in the Week 2 HTML skeleton.
- `width=device-width`: use the actual device width.
- `initial-scale=1`: do not zoom in or out on first load.



# Media Query Syntax

```
1  /* Default: desktop layout */
2  .dashboard {
3      display: flex;
4      flex-direction: row;
5      gap: 12px;
6  }
7
8  /* Tablet and below */
9  @media screen and (max-width: 768px) {
10     .dashboard { flex-direction: column; }
11 }
12
13 /* Phone */
14 @media screen and (max-width: 480px) {
15     .card { padding: 10px; }
16 }
```

# Responsive KPI Dashboard (Full Example)

```
1 .dashboard {  
2     display: flex;  
3     flex-wrap: wrap;  
4     gap: 12px;  
5 }  
6 .card {  
7     flex: 1 1 220px;  
8     padding: 14px;  
9     border: 1px solid #d4d4d4;  
10    border-radius: 10px;  
11    background: #ffffff;  
12 }  
13 @media screen and (max-width: 640px) {  
14     .dashboard { flex-direction: column; }  
15 }
```

# Responsive Units

```
1 body { font-size: 16px; }
2 h1   { font-size: 2rem; } /* 32px relative to root */
3 p    { font-size: 1rem; } /* always 16px */
4
5 .card { max-width: 100%; } /* never overflow container */
6 img   { max-width: 100%; height: auto; } /* responsive image */
```

- rem: relative to root font size – consistent across components.
- % and vw: relative to container / viewport – flexible.
- Avoid fixed px widths on outer containers.



## Further Reading (Part 4)

### Optional – self-study handout

- Mobile-first strategy: start with min-width, expand upward.
- CSS custom properties (–variables) for consistent theming.
- Bootstrap 5: a CSS framework built on Flexbox and Grid.
- Source: MDN – *Responsive Web Design Basics; Using Media Queries*.



## Lab 4: Responsive Dashboard

### Hands-on practice

Extend your Lab 3 dashboard:

- Add the viewport meta tag in the HTML.
- Add a @media rule: below 768px, cards stack vertically.
- Resize the browser window and verify the transition.
- *Finished early?* Add a second breakpoint at 480px that reduces card padding.
- *You may label cards with DS/AI metrics: Accuracy, Loss, F1 Score, Latency.*



# Review & Summary

---



## Quick MCQ Review

1. Which property adds spacing *inside* the border? **padding**
2. A div has width:200px, padding:10px, border:2px, margin:5px. Total width? **234px**
3. Which selector wins: .title or #header? **#header**
4. Media query for screens at least 800px wide? @media (min-width: 800px)
5. Flexbox property for main-axis distribution? **justify-content**
6. Why use border-box? **padding and border included in declared width**



# Self-Check Checklist

- ☐ CSS linked via `<link rel="stylesheet" href="style.css">`
- ☐ Global box-sizing: `border-box` reset applied
- ☐ Viewport meta tag present in every HTML file
- ☐ Flexbox used for card and navigation layouts
- ☐ At least one `@media` rule tested at 768px
- ☐ No fixed-pixel widths on outer containers





## Summary and Next Session

- **CSS** is a rule system driven by selectors, specificity, and cascade.
- **Box Model** is the foundation of all spacing – content, padding, border, margin.
- **Flexbox** is the go-to tool for one-dimensional card and navigation layouts.
- **Media queries** make your page adapt gracefully to any screen size.
- These techniques directly power DS/AI dashboards and reporting interfaces.
- Next: **JavaScript** brings interaction to the styled page (Week 4).

## Questions & Answers